EXPERIMENT 7

Shravanya Andhale

D20A-02

Comparison of GPT and BERT Models

AIM:

To explore and compare GPT and BERT Transformer models by performing text generation, masked word prediction, and text classification tasks, and understand their architectural differences and applications in Natural Language Processing.

THEORY:

Transformer models are deep learning architectures designed to process sequential data, particularly natural language. They use an attention mechanism to understand relationships between different words in a sentence. Unlike traditional recurrent neural networks, Transformers can process tokens in parallel during training and capture long-range dependencies more effectively.

GPT and BERT are two important pretrained Transformer-based language models. Both are trained on large text datasets, but they use different architectures and training objectives. These differences make them suitable for different Natural Language Processing tasks.

1. GPT (Generative Pre-trained Transformer):

GPT is based on a decoder-only Transformer architecture. It uses autoregressive language modeling, where the model predicts the next token based on the tokens that appeared before it.

For example, given the input sentence "Artificial Intelligence is", GPT predicts the next likely token and continues generating text one token at a time. This process continues until the model reaches a stopping condition or the specified maximum output length.

GPT is particularly suitable for generative tasks such as:

- Text and paragraph generation.
- Question answering.
- Creative writing.
- Summarization.
- Code generation.
- Conversational AI.

GPT uses causal attention, which prevents the model from accessing future tokens while predicting the next token.

2. BERT (Bidirectional Encoder Representations from Transformers):

BERT is based on an encoder-only Transformer architecture. Unlike GPT, BERT uses bidirectional context, meaning it considers both the left and right sides of a word to understand its meaning.

BERT was originally trained using objectives such as Masked Language Modeling and Next Sentence Prediction in its original training setup.

In Masked Language Modeling, certain words in a sentence are replaced with a special `[MASK]` token. The model predicts the missing word using the surrounding context.

For example:

```
The capital of France is [MASK].
```

BERT uses the context of the sentence to predict the missing word, which is likely to be "Paris".

BERT is useful for:

- Text classification.
- Sentiment analysis.
- Named Entity Recognition.
- Question answering.
- Masked token prediction.
- Text understanding tasks.

3. GPT and BERT Comparison:

Feature	GPT	BERT
Architecture	Decoder-only	Encoder-only
Context	Left-to-right	Bidirectional
Main objective	Next-token prediction	Masked language modeling in original BERT
Main purpose	Text generation	Text understanding

Typical tasks	Text generation, conversation	Classification, masked prediction

4. Text Generation and Masked Token Prediction:

In this experiment, GPT is used to generate text based on an input prompt. The generated response is produced by predicting successive tokens. BERT is used to predict masked words in a sentence by analyzing the surrounding context. These tasks demonstrate the different capabilities of the two models.

5. Text Classification:

BERT can also be used for text classification by passing the contextual representation of a sentence through a classification layer. This layer predicts a category or label based on the input text. Examples include positive or negative sentiment classification and topic categorization.

By implementing these tasks, the experiment demonstrates how Transformer-based models can perform both generative and language-understanding tasks.

CODE AND OUTPUT:

Experiment 7 - Transformer Models: GPT and BERT

```
!pip -q install transformers torch
```

```
import torch, pandas as pd
from transformers import pipeline
device = 0 if torch.cuda.is_available() else -1
print("Pipeline device:", "GPU" if device == 0 else "CPU")
```

```
Pipeline device: CPU
```

```
# GPT-2 text generation: three prompts in a cricket coaching context.
generator = pipeline("text-generation", model="gpt2", device=device)
gpt_prompts = [
    "A beginner cricket batter can improve their balance by",
    "A coach reviewing a cover drive should first observe",
    "A useful 15-minute batting practice session includes"
]
gpt_outputs=[]
for prompt in gpt_prompts:
    out=generator(prompt, max_new_tokens=55, do_sample=True, temperature=0.8, top_k=50,
                  pad_token_id=generator.tokenizer.eos_token_id)[0]["generated_text"]
    gpt_outputs.append({"Prompt":prompt,"GPT-2 output":out})
    print("\nPROMPT:",prompt,"\nOUTPUT:",out)
gpt_df=pd.DataFrame(gpt_outputs)
display(gpt_df)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 665/665 [00:00<00:00, 61.0kB/s]
model.safetensors: reconstructing file: 100% 548MB / 548MB, 48.1MB/s
model.safetensors: downloading bytes: 474MB, 40.2MB/s
Loading weights: 100% 148/148 [00:00<00:00, 1713.18it/s]
generation_config.json: 100% 124/124 [00:00<00:00, 3.31kB/s]
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 523B/s]
vocab.json: 100% 1.04M/1.04M [00:00<00:00, 7.25MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 5.19MB/s]
```

```
PROMPT: A beginner cricket batter can improve their balance by
OUTPUT: A beginner cricket batter can improve their balance by being faster and sharper than a beginner. This method is very similar to Kullen batsman who hits an inside-out swing.

Another method of playing a beginner cricket batter is to use a different batting style altogether. If you know how to play cricket, you

PROMPT: A coach reviewing a cover drive should first observe
OUTPUT: A coach reviewing a cover drive should first observe the situation and make a decision as to what the best strategy for the player should be. A coach reviewing a cover drive should first observe the situation and make a decision as to what the best strategy for the

The best cover drive is a short one

PROMPT: A useful 15-minute batting practice session includes
OUTPUT: A useful 15-minute batting practice session includes a variety of games from batting practice to batting practice. The sessions are also designed to give you the chance to interact with other players in a non-stadium setting.


There is no charge to the attendance at the event.

The event is free and open
```

	Prompt	GPT-2 output
0	A beginner cricket batter can improve their ba...	A beginner cricket batter can improve their ba...
1	A coach reviewing a cover drive should first o...	A coach reviewing a cover drive should first o...
2	A useful 15-minute batting practice session in...	A useful 15-minute batting practice session in...

```
# Observe the effect of decoding parameters on the same prompt.
prompt="A cricket coach gives feedback by"
for temp in [0.3, 0.8, 1.2]:
    out=generator(prompt,max_new_tokens=45,do_sample=True,temperature=temp,top_k=50,
                  pad_token_id=generator.tokenizer.eos_token_id)[0]["generated_text"]
    print(f"\nTemperature={temp}\n{out}")
```

```
Temperature=0.3
A cricket coach gives feedback by phone to his players.

"I've been told that if you don't get the ball, you don't get the ball. I've been told that if you don't get the ball, you don't get

Temperature=0.8
A cricket coach gives feedback by himself on the outcome of a Test match. Photo: Reuters

A cricket coach gives feedback by himself on the outcome of a Test match. Photo: Reuters

A cricket coach gives feedback by himself on the outcome of

Temperature=1.2
A cricket coach gives feedback by signing and fielding. "You have to think that way of doing things," he added, adding that "all the things you do are important too."

In other words, you need to take steps now to create pressure
```

```python
# BERT masked-token prediction.
fill_mask = pipeline("fill-mask", model="bert-base-uncased", device=device)
masked = "A cricket batter should maintain [MASK] while playing a shot."
bert_mask_outputs = fill_mask(masked)
print("Masked sentence:", masked)
for item in bert_mask_outputs[:5]:
    print(item["sequence"], "| score:", round(item["score"],4))
```

```
config.json: 100% 570/570 [00:00<00:00, 21.1kB/s]
model.safetensors: reconstructing file: 100% 440MB / 440MB, 40.1MB/s
model.safetensors: downloading bytes: 415MB, 37.0MB/s
Loading weights: 100% 202/202 [00:00<00:00, 4094.44it/s]
[transformers] BertForMaskedLM LOAD REPORT from: bert-base-uncased
Key                          | Status     |  |
-----------------------------+------------+--+-
cls.seq_relationship.weight | UNEXPECTED |  |
bert.pooler.dense.bias      | UNEXPECTED |  |
bert.pooler.dense.weight    | UNEXPECTED |  |
cls.seq_relationship.bias   | UNEXPECTED |  |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.58kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 3.54MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 12.7MB/s]
Masked sentence: A cricket batter should maintain [MASK] while playing a shot.
a cricket batter should maintain balance while playing a shot. | score: 0.1222
a cricket batter should maintain control while playing a shot. | score: 0.0953
a cricket batter should maintain composure while playing a shot. | score: 0.0891
a cricket batter should maintain calm while playing a shot. | score: 0.0874
a cricket batter should maintain distance while playing a shot. | score: 0.0557
```

```python
# A second BERT-family understanding task: sentiment classification.
# This is a fine-tuned DistilBERT classifier, while BERT-base above performs masked-token prediction.
sentiment = pipeline("sentiment-analysis",
                     model="distilbert/distilbert-base-uncased-finetuned-sst-2-english",
                     device=device)
sentences = [
    "The coaching session was helpful and encouraging.",
    "The instructions were confusing and frustrating."
]
for sentence in sentences:
    print(sentence, "=>", sentiment(sentence)[0])
```

```
config.json: 100% 629/629 [00:00<00:00, 20.2kB/s]
model.safetensors: reconstructing file: 100% 268MB / 268MB, 25.4MB/s
model.safetensors: downloading bytes: 250MB, 23.2MB/s
Loading weights: 100% 104/104 [00:00<00:00, 4101.39it/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 2.70kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 11.7MB/s]
The coaching session was helpful and encouraging. => {'label': 'POSITIVE', 'score': 0.9998587369918823}
The instructions were confusing and frustrating. => {'label': 'NEGATIVE', 'score': 0.9995136260986328}
```

```
comparison = pd.DataFrame([
    ["Architecture","Decoder-only","Encoder-only (BERT); this notebook also uses a fine-tuned DistilBERT classifier"],
    ["Primary purpose","Autoregressive text generation","Language understanding / contextual representations"],
    ["Training objective","Next-token prediction","Masked-token prediction (pre-training)"],
    ["Input/output","Prompt → generated continuation","Masked sentence → token candidates; text → class label"],
    ["Strength","Flexible continuation and generation","Contextual understanding for supported tasks"],
    ["Limitation","May continue incoherently or invent claims","Not a general-purpose long-form text generator"]
],columns=["Parameter","GPT-2","BERT / BERT-family"])
display(comparison)
```

	Parameter	GPT-2	BERT / BERT-family
0	Architecture	Decoder-only	Encoder-only (BERT); this notebook also uses a...
1	Primary purpose	Autoregressive text generation	Language understanding / contextual representa...
2	Training objective	Next-token prediction	Masked-token prediction (pre-training)
3	Input/output	Prompt → generated continuation	Masked sentence → token candidates; text → cla...
4	Strength	Flexible continuation and generation	Contextual understanding for supported tasks
5	Limitation	May continue incoherently or invent claims	Not a general-purpose long-form text generator

https://colab.research.google.com/drive/1SW93mnn3QjDz3T5eEt4maQAn5HFaNZ27?usp=sharing

CONCLUSION:

The experiment demonstrated the working and applications of GPT and BERT models. GPT was used for autoregressive text generation, while BERT was used for masked token prediction and text classification. The comparison highlighted the differences between decoder-only and encoder-only Transformer architectures and showed how different pretrained language models are selected for different NLP applications.